

```

const TelegramBot = require('node-telegram-bot-api');
const express = require('express');
const cors = require('cors');

const BOT_TOKEN = process.env.BOT_TOKEN;

if (!BOT_TOKEN) {
  console.error('ERROR: BOT_TOKEN environment variable is not set!');
  process.exit(1);
}

const app = express();
app.use(cors());
app.use(express.json());

// Start express FIRST before bot
const PORT = process.env.PORT || 8080;
const server = app.listen(PORT, '0.0.0.0', () => {
  console.log(`Maher Bot server running on port ${PORT}`);
});

// Then start bot
const bot = new TelegramBot(BOT_TOKEN, { polling: true });
console.log('Telegram bot polling started...');

let messages = [];
let analysis = {
  subjects: [],
  assignments: [],
  exams: [],
  notes: [],
  lastUpdated: null
};

const ASSIGNMENT_WORDS = ['assignment', 'homework', 'submit', 'submission', 'due'];
const EXAM_WORDS = ['exam', 'test', 'quiz', 'midterm', 'final', 'viva', 'practica'];
const NOTE_WORDS = ['lecture', 'notes', 'slide', 'pdf', 'chapter', 'topic', 'syllabus'];
const DATE_PATTERN = /(\d{1,2}[\//\-\.] \d{1,2}[\//\-\.] \d{2,4})|(\d{1,2} s*(jan|feb|march|apr|may|jun|jul|aug|sep|oct|nov|dec)) /;
const SUBJECT_PATTERN = /(maths?|physics|chemistry|biology|english|history|computer science)/;

function extractSubject(text) {
  const found = text.match(SUBJECT_PATTERN);
  return found ? [...new Set(found.map(s => s.charAt(0).toUpperCase() + s.slice(1)))].join(' ') : null;
}

```

```

function extractDate(text) {
  const found = text.match(DATE_PATTERN);
  return found ? found[0] : null;
}

function classifyMessage(text) {
  const lower = text.toLowerCase();
  if (ASSIGNMENT_WORDS.some(w => lower.includes(w))) return 'assignment';
  if (EXAM_WORDS.some(w => lower.includes(w))) return 'exam';
  if (NOTE_WORDS.some(w => lower.includes(w))) return 'note';
  return null;
}

function runAnalysis() {
  const assignmentsMap = {};
  const examsMap = {};
  const notesArr = [];
  const subjectsSet = new Set();

  for (const msg of messages) {
    const type = classifyMessage(msg.text);
    const subjects = extractSubject(msg.text);
    const date = extractDate(msg.text);
    subjects.forEach(s => subjectsSet.add(s));

    if (type === 'assignment') {
      const key = (subjects[0] || 'General') + '_' + (date || msg.text.slice(0, 2));
      if (!assignmentsMap[key]) {
        assignmentsMap[key] = {
          subject: subjects[0] || 'General',
          text: msg.text.slice(0, 120),
          deadline: date || 'No date mentioned',
          group: msg.groupName,
          time: msg.time,
          done: false
        };
      }
    }
    else if (type === 'exam') {
      const key = (subjects[0] || 'General') + '_exam_' + (date || '');
      if (!examsMap[key]) {
        examsMap[key] = {
          subject: subjects[0] || 'General',
          text: msg.text.slice(0, 120),
          date: date || 'Date TBC',
          group: msg.groupName,
          time: msg.time
        };
      }
    }
  }
}

```

```

        };
    }
    } else if (type === 'note') {
        notesArr.push({
            subject: subjects[0] || 'General',
            text: msg.text.slice(0, 100),
            group: msg.groupName,
            time: msg.time
        });
    }
}

analysis = {
    subjects: [...subjectsSet],
    assignments: Object.values(assignmentsMap).slice(0, 20),
    exams: Object.values(examsMap).slice(0, 10),
    notes: notesArr.slice(0, 15),
    totalMessages: messages.length,
    lastUpdated: new Date().toISOString()
};

}

bot.on('message', (msg) => {
    if (!msg.text) return;
    const chatType = msg.chat.type;
    if (chatType !== 'group' && chatType !== 'supergroup') return;

    messages.push({
        id: msg.message_id,
        text: msg.text,
        groupName: msg.chat.title || 'Unknown Group',
        groupId: msg.chat.id,
        sender: msg.from ? msg.from.first_name : 'Unknown',
        time: new Date(msg.date * 1000).toISOString()
    });

    if (messages.length > 500) messages = messages.slice(-500);
    if (messages.length % 5 === 0) runAnalysis();
});

bot.onText(/\/start/, (msg) => {
    bot.sendMessage(msg.chat.id, '👋 Hi! I am Maher College Bot!\n\nAdd me to your');
});

bot.onText(/\/status/, (msg) => {
    bot.sendMessage(msg.chat.id, `🇮🇵 Status:\n💬 Messages: ${messages.length}\n📁 `);
});

```

```

bot.onText(/\//summary/, (msg) => {
  let text = '📖 Summary:\n\n';
  if (analysis.assignments.length) {
    text += `📅 ASSIGNMENTS:\n`;
    analysis.assignments.slice(0, 3).forEach(a => { text += `• ${a.subject} - ${a
  }
  if (analysis.exams.length) {
    text += `\n📅 EXAMS:\n`;
    analysis.exams.slice(0, 3).forEach(e => { text += `• ${e.subject} - ${e.date}
  }
  if (!analysis.assignments.length && !analysis.exams.length) {
    text += 'Nothing detected yet. Make sure I am in your college groups!';
  }
  bot.sendMessage(msg.chat.id, text);
});

```

```

app.get('/', (req, res) => res.json({ status: 'Maher Bot running ✅', messages: m
app.get('/api/analysis', (req, res) => { if (messages.length > 0 && !analysis.las
app.get('/api/messages', (req, res) => res.json({ total: messages.length, recent:
app.post('/api/mark-done', (req, res) => { const { index } = req.body; if (analys
app.get('/api/refresh', (req, res) => { runAnalysis(); res.json({ success: true,

```

```

bot.on('polling_error', (error) => console.error('Polling error:', error.message)
process.on('uncaughtException', (err) => console.error('Uncaught exception:', err

```